UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Labyrinth escaper

Berinde Codrin, Daisa Petru

Technical University of Cluj-Napoca

Faculty of Automatics and Computer Science

Specialisation: *Automatics and Applied Informatics*

Year of study: *1*

Date: *January 2024*

Contents

1 Project outline **3**

2 Maze generation **3**
2.1 Theoretical considerations . . . 3
2.1.1 The characteristics of the maze . . . 3
2.1.2 The generating algorithm . . . 4
2.1.3 DFS algorithm using the stack data structure . . . 5
2.2 Implementation . . . 7
2.2.1 The labelling of the directions . . . 7
2.2.2 The stack . . . 7
2.2.3 The maze matrix . . . 9
2.2.4 The visited cells . . . 9
2.2.5 Other preparatory functions . . . 10
2.2.6 Choosing the next cell . . . 11
2.2.7 The DFS function . . . 13
2.2.8 The main function . . . 14

3 Maze solving **16**
3.1 Theoretical considerations . . . 16
3.2 Implementation . . . 17
3.2.1 The queue . . . 17
3.2.2 The parent matrix . . . 18
3.2.3 Other preparatory functions . . . 18
3.2.4 The BFS function . . . 19
3.2.5 Recovering the path . . . 19
3.2.6 The main function . . . 22

4 Simulation **22**
4.1 Creating the environment . . . 22
4.2 The robot . . . 24
4.2.1 Design . . . 24
4.2.2 Control . . . 26

Appendices **30**
A - *Source and header files* . . . 30
B - *A challenge for the reader* . . . 39

1 Project outline

The *Labyrinth escaper* is a project developed for the *Computer Programming and Programming Languages* first-year course in the *Automation and Applied Informatics* specialization of the Technical University of Cluj-Napoca.

The objectives of this project is to randomly generate mazes, to find a solution path for each of them, and finally, to graphically implement a 3D simulation in Webots. The simulation presents a robot that fastly runs through the maze, from start to finish.

The project, entirely programmed in the C language, is more complex than it may seem at first sight. We hope the reader will understand its complexity from the following documentation.

Before diving into the technical aspects, we would like to thank our professor, Teodora Sanislav, for all the support and guidance she offered us in the field of C programming.

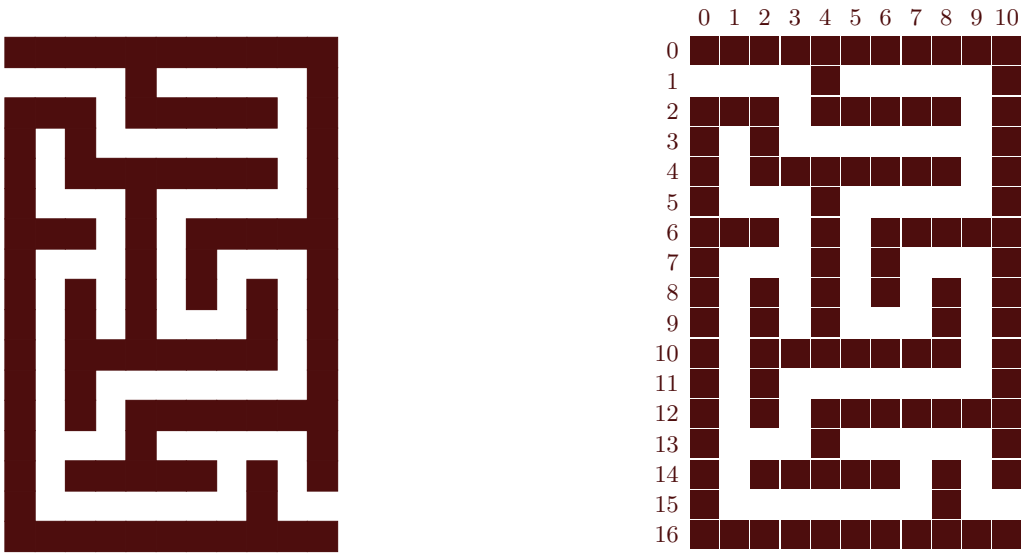
2 Maze generation

2.1 Theoretical considerations

2.1.1 The characteristics of the maze

The first necessary step of this project is to have a maze. Figure 1a) depicts the general characteristics of the type of the maze we will generate and use, on an arbitrary configuration. It is easily noticeable that

1. The thickness of the walls equals the thickness of the corridors.
2. The entrance of the maze is the upper-leftmost piece of corridor and the exit from the maze is the lower-rightmost one.
3. The only breaks in the outer walls of the maze are at the entrance and the exit.



a) b)

Figure 1: The characteristics of the maze

The maze will be manipulated throughout this project as a matrix, behaviour which is emphasized in Figure 1b), where the arbitrary maze we have chosen is incorporated in a 17×11 matrix. In the figure, the walls were gently separated for visual reasons into what will be referred to from now on as *cells*. Each piece of wall and corridor is fitted into a cell. Therefore, each cell of the maze can be of two types: wall or corridor. Remark the following

1. The labelling convention of the cells: for a `height` $\times$ `width` maze the cells are named (i, j), where $i \in \{0, \ldots, \texttt{height} - 1\}$, $j \in \{0, \ldots, \texttt{width} - 1\}$, $(0, 0)$ being the upper-leftmost one. Also, the first index of a cell is the row of the matrix, while the second one is the column.

2. Another aspect will turn out to be extremely useful when generating the maze. Observe that in the maze, all the cells having both indices even are walls. All the cells having both indices odd are corridor. We will refer to these cells as *always-free cells*. However, the cells having indices of differrent parity are the only ones that determine the configuration of the maze and of the path from the entrance to the exit. Just because of the fact that they differ from a maze to another, these cells will be called *versatile cells*.

3. Taking a look to the rows consecutively, one can easily see that the maze consists of alternances of horizontal blocks of walls which separate horizontal corridors from each other and ends with the last horizontal block of walls. Taking a look on the columns, the same behaviour is observed, but with vertical blocks of walls separating vertical corridors. For this reason, every maze of this type has an odd number of rows and an odd number of columns. A maze of this type, having even dimensions, cannot be built.

2.1.2 The generating algorithm

In order to make our project as diversified as possible, we implemented an algorithm that generates mazes of any size, given as input by the user, and are built based on a number called *seed*, such that for a given size and seed, we always get the same maze.

The algorithm we have chosen to generate the maze is the classic depth-first-search (DFS). First we consider an `height` $\times$ `width` grid (which will later become our maze), where `height` and `width` are odd numbers.

Initially, the maze has walls placed in all its versatile cells. This situation is depicted in Figure 2a) for a 7×7 grid. The figure presents all the steps the algorithm makes while generating the maze. It starts from the *always-free cell* (`xstart`, `ystart`), in our example $(1, 1)$, and gradually visits the entire matrix, following the next steps

1. It evaluates from the current cell it is situated in (marked yellow in Figure 2), what are the possible directions it can go to. The valid options (marked with red arrows in Figure 2) are the neighbouring *always-free cells*. Since a *versatile cell* separates a unique pair of *always-free cells*, it is useless that the algoritm wastes time and resources to visit also the *versatile cell* when, instead, it can directly jump to the *always-free cell*. However, for a neighbouring *always-free cell* to represent a valid option, it must not have already been visited by the algorithm (visited cells are marked blue in Figure 2), otherwise it can enter infinite loops.

2. It randomly chooses one of the valid directions and goes into that cell.

3. It erases the wall between the cell it has prevoiusly been to an the one it is currently in.

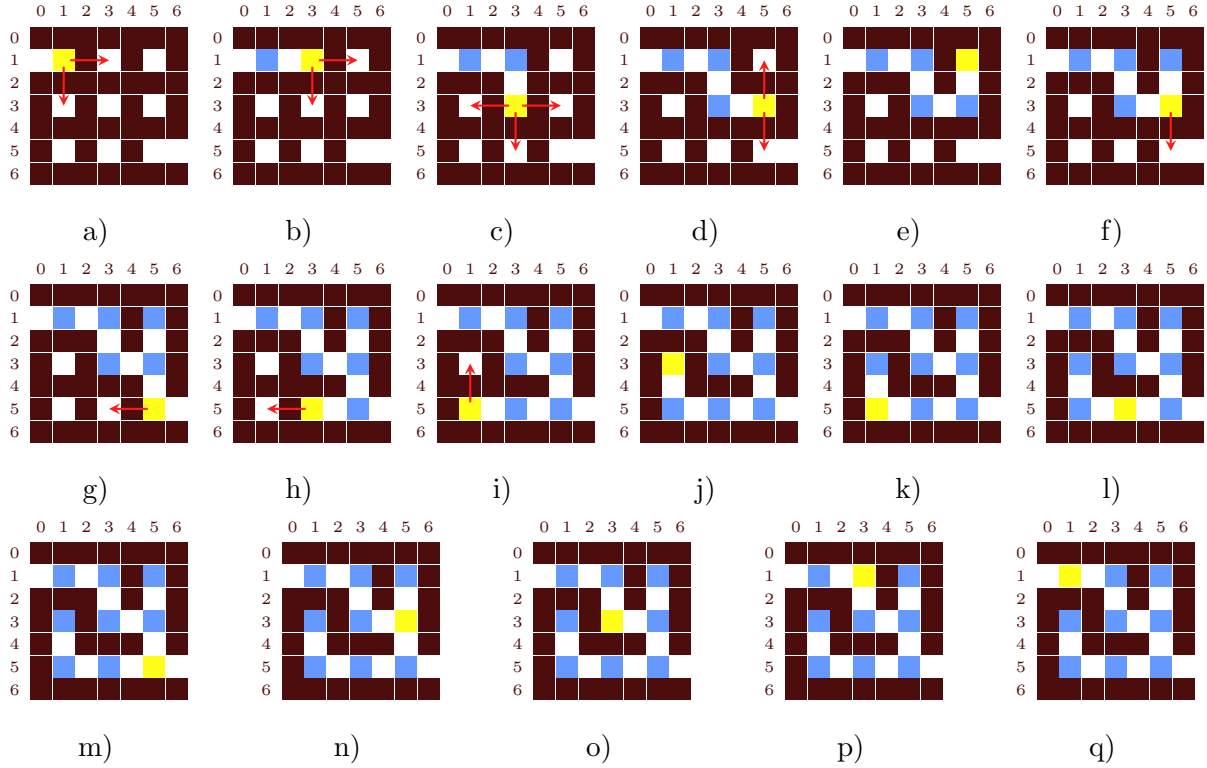


Figure 2: Maze generation using DFS

4. If from one cell there is no available direction to go to (step e) from Figure 2), it returns to the previous cells it has been to in reverse order, until it reaches one that has other valid directions (step f)).

This procedure is repeated over and over again until the algorithm returns to the starting cell. At this point, it is guaranteed that

1. all *always-free cells* of the grid have been visited (and automatically the entire maze), because otherwise the algorithm would not yet return to the starting cell, having other cells with valid directions to go to.

2. there is a unique path from any cell to any other cell of the grid, because the algorithm never breaks a wall to an already-visited cell, and therefore it creates no loops.

Considering that, we conclude that this is a satisfactory algorithm to use for our project. In what follows, we will describe its actual implementation.

2.1.3 DFS algorithm using the stack data structure

Recall from the working principle of the algorithm that, provided we arrive to a cell with no possible direction to go to, we have to return to the previous cell. Therefore, we must not only memorize all the cells we have prevoiusly been to, but we should also be able to access them in the reverse order they have been added, or in other words, on the principle *last-in-first-out* (LIFO) (from a cell, if we have nowhere to go to, we should be able to **first** access the **last** cell we have previously visited). Such a data structure is called a *stack* and it possesses many

advantages, such as the possibility to allocate dinamically the memory for it. A stack can be visually represented as a box, in which we can put objects one on top of the other, but we can always access only the object at the top.

Figure 3 presents an exemplification of the functioning of the stack on the same 7×7 maze as in Figure 2. From lack of space, only some of the key steps were selected to be drawn.

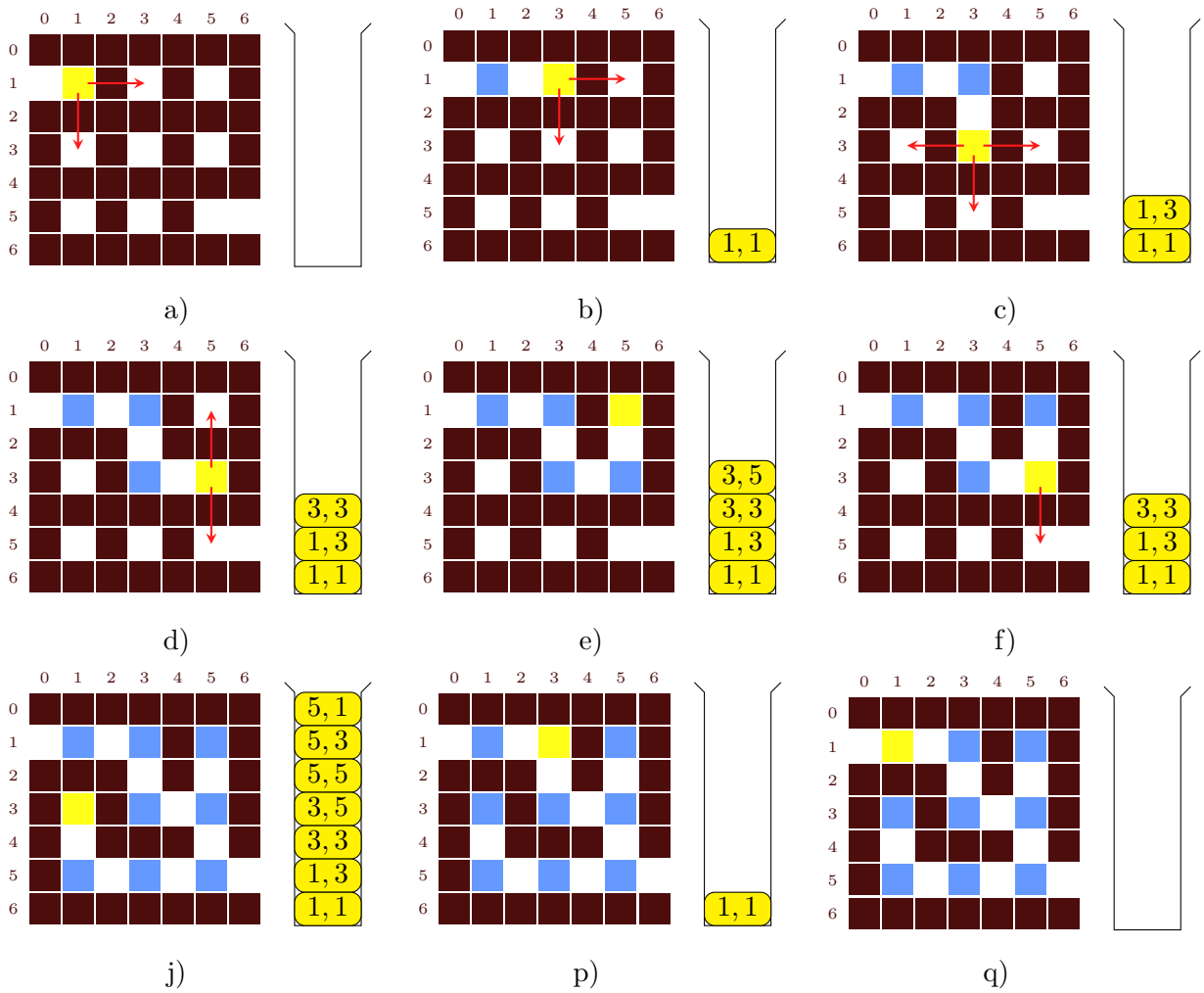


Figure 3: DFS algorithm using the stack data structure

Follow the steps the algorithm takes

- Step a) - we begin in cell $(1, 1)$ with the stack being empty and evaluate the possible directions to go to, which are right and down. We randomly choose one direction to go to, say, right, and add the current cell to the stack.
- Step b) - we are in cell $(1, 3)$ and the possible directions are right and down. We choose to go down and add the current cell to the stack.
- Step c) - we are in cell $(3, 3)$ and the possible directions are right, down and left. We choose to go down and add the current cell to the stack.

- Step d) - we are in cell $(3, 5)$ and the possible directions are up and down. We choose to go up and add the current cell to the stack.
- Step e) - we are in cell $(1, 5)$. There is no valid direction to go to from here. We do not add this cell to the stack because it is a dead end, but we return to the cell at the top of the stack, which is $(3, 5)$, and remove it from the stack

  ...
- Step j) - we are in cell $(3, 1)$ where we got stuck again. We return to the cell at the top of the stack, which is $(5, 1)$, and remove it from the stack.

  ...
- Step p) - we are in cell $(1, 3)$ but we have nowhere to go to, hence we return to the cell at the top of the stack, $(1, 1)$, and remove it from the stack.
- Step q) - we have returned to the starting cell, $(1, 1)$, with the stack being empty. Hence the algorithm is over. The maze has been fully visited and paths were made from the starting cell to all its cells.

2.2 Implementation

Most of the functions related to the generation of the maze were defined in a separate file called *maze.c*.

2.2.1 The labelling of the directions

We considered the following labelling of the directions, relative to a cell, which are presented in Figure 4.

We also defined a global matrix of relative direction in order to pass easily from a cell to another, only by applying the `relative_dir` matrix to the current cell indices.

```
static int relative_dir[4][2] = {{0, 2}, {2, 0}, {0, -2}, {-2, 0}};
```

The first dimension of the `relative_dir` matrix represents the direction in which we want to go and the second dimension represents the index of the cell that we are currently modifying: `relative_dir[x][0]` modifies the first index of the cell in the direction `x` and `relative_dir[x][1]` modifies the second index of the cell in the direction `x`.

2.2.2 The stack

We have manually implemented the stack data structure by defining a new `struct` that contains all the necessary data for a cell. In our structure we must have the x and y coordinates of the cell, but also a reference, called `link`, to the previous cell, because each time we want to remove a cell we must know which was the previous one.

```
typedef struct cell {
  int x, y;
  struct cell *link;
} cell;
```

Listing 1: The `cell` data type

Label	Direction	Indices
0	right	second index of the cell increases
1	down	first index of the cell increases
2	left	second index of the cell decreases
3	up	first index of the cell decreases

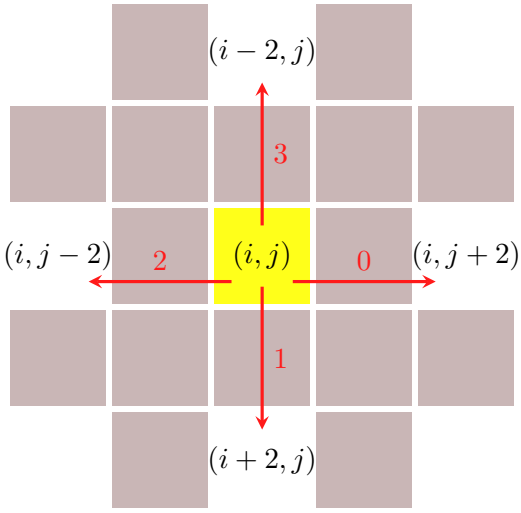


Figure 4: The labelling of the directions

By doing this, we have to memorize only the cell at the top of the stack, denoted as the `top` cell, since we can always access the previous one.

We have defined all the necessary functions related to the stack in a separate file called *structures.c*. Listing 2 presents the basic functions that we will use.

The function `stack_add()` adds an element in the stack. It requires a pointer to the old top and the coordinates of the new cell as parameters and returns a pointer to the new top of the stack.

The function `rem()` removes the top element of the stack. It requires a pointer to that element as parameter and returns pointer to the new top. The first element introduced in the stack must be properly initialized, its `link` member must be the `NULL` pointer, such that when function `rem()` removes the last element of the stack and it becomes empty, the function will return `NULL`.

Finally, the function `is_empty()` checks whether the stack is empty.

```
cell *stack_add(cell *old_cell, int cell_x, int cell_y) {
  cell *new_cell;
  new_cell = (cell *)malloc(sizeof(cell));
  new_cell->x = cell_x;
  new_cell->y = cell_y;
  new_cell->link = old_cell;
  return new_cell;
}

cell *rem(cell *old_cell) {
  cell *new_cell;
  new_cell = old_cell->link;
  free(old_cell);
  return new_cell;
}
```

```c

int is_empty(cell *top_cell) {
  return (top_cell == NULL);
}
```

Listing 2: The basic functions for the stack

2.2.3 The maze matrix

As it was previously discussed, we need a matrix to store whether in a cell is wall or corridor. This matrix is sugestively named `maze` and is initialized using the `initialize_maze()` function, presented in Listing 3. The initialization occurs according to relation 1.

$$\texttt{maze[i][j]} = \begin{cases} 0, \text{ if cell } (i,j) \text{ is corridor} \\ \\ 1, \text{ if cell } (i,j) \text{ is wall} \end{cases} \tag{1}$$

```c
int **initialize_maze(int width, int height) {
  int **maze = (int **)calloc(sizeof(int *), height);
  for(int row = 0; row < height; row++) {
    *(maze + row) = (int *)calloc(sizeof(int), width);
    for(int column = 0; column < width; column++) {
      *(*(maze + row) + column) = ((row % 2 == 1) && (column % 2 == 1)) ? 0 : 1;
    }
  }
  *(*(maze + 1)) = 0;
  *(*(maze + height - 2) + width - 1) = 0;
  return maze;
}
```

Listing 3: Initializing the `maze` matrix

The function takes as parameters the dimensions of the maze and returns a pointer to the matrix `maze`.

2.2.4 The visited cells

For keeping track of the visited cells we also defined a matrix.

$$\texttt{visited[i][j]} = \begin{cases} 0, \text{ if the cell } (i,j) \text{ has not yet been visited} \\ \\ 1, \text{ if the cell } (i,j) \text{ has been visited} \end{cases}$$

The function `initialize_visited()` initializes this matrix and returns a pointer to it. The function `calloc()` already initializes all the elements of the matrix with the value 0.

```c
int **initialize_visited(int width, int height) {
  int **visited = (int **)calloc(sizeof(int *), height);
  for(int row = 0; row < height; row++) {
    *(visited + row) = (int *)calloc(sizeof(int), width);
  }
  return visited;
}
```

Listing 4: Initializing the `visited` matrix

2.2.5 Other preparatory functions

Recall that we will have to randomly choose a direction to go to, therefore a function which chooses a random number between 0 and the number of valid neighbouring cells (which is variable, transmitted as the parameter `max_dir` to the function) is needed.

```c
int random_dir(int max_dir) {
  return rand() % max_dir;
}
```

Listing 5: The `random_dir()` function

We need a function that decides whether the cell we want to go to does not exceed the boundaries of the maze, because the DFS algorithm blindly test all the cells around one cell. If not carefully implemented, it may choose to go to a cell that does not even exist, not being aware that the current cell is on the edge of the maze.

```c
int in_matrix(int x, int y, int xmax, int ymax) {
  return (x >= 0) && (x < xmax) && (y >= 0) && (y < ymax);
}
```

Listing 6: The `in_matrix()` function

Another function was defined, that eliminates the suitable wall after moving to a cell. The function `smash_walls()` receives as parameters the coordinates `x` and `y` of the cell we have moved to, the `direction` we applied to get into this cell and the pointer to the `maze` matrix.

```c
void smash_walls(int x, int y, int direction, int **maze) {
  switch(direction) {
    case 0: //we came from left
      *(*(maze + x) + y - 1) = 0;
      break;
    case 1: //we came from above
      *(*(maze + x - 1) + y) = 0;
      break;
    case 2: //we came from right
      *(*(maze + x) + y + 1) = 0;
      break;
    case 3: //we came from below
      *(*(maze + x + 1) + y) = 0;
      break;
  }
}
```

Listing 7: The `smash_walls()` function

Lastly, we need a function to free the memory that was allocated for the auxiliary matrix `visited`, used in the process of maze creation. We defined a function that frees the memory of any given matrix (because we will use this function even for other matrices, later) of arbitrary height `height`. The function takes as parameter a pointer to the matrix pointer, because the matrix pointer is modified internally.

```c
void clear_mem(int height, int ***matrix) {
  for(int row = 0; row < height; row++) {
    free(*(matrix + row));
  }
  free(matrix);
}
```

Listing 8: The `clear_mem()` function

2.2.6 Choosing the next cell

One of the surprisingly tricky part of the algoritm is, when being in a cell, choosing a neighbouring cell to go to, or, equivalently, choosing one of the four directions. The direction must be random and must be a valid one, valid meaning that the cell in that direction is neither outside the maze, nor yet visited. The function `direction_choice()` receives as parameters pointers to the coordinates of the current cell, which will be modified internally into the coordinates of the cell we moved to, the dimensions of the maze in the form `xmax` (height) and `ymax` (width). The function also returns an integer: -1, if there is no valid direction to go to, or the number of the randomly-chosen direction, if there is one or more valid directions.

```
1  int direction_choice(int *x, int *y, int xmax, int ymax, int **visited) {
2    int option_mask = 0, option_number = 0, choice = 0, option_counter = 0, chosen_direction = 4;
3
4    for(int direction = 0; direction < 4; direction++) {
5      option_mask *= 2;
6      if(in_matrix(*x + relative_dir[direction][0], *y + relative_dir[direction][1], xmax, ymax) &&
7         !(*(*(visited + *x + relative_dir[direction][0]) + *y + relative_dir[direction][1]))) {
8        option_number++;
9        option_mask += 1;
10     }
11   }
12   if(option_number == 0) {
13     return -1;
14   }
15
16   choice = random_dir(option_number);
17   choice++;
18   while(option_counter != choice) {
19     if(option_mask % 2 == 1) {
20       option_counter++;
21     }
22     chosen_direction--;
23     option_mask /= 2;
24   }
25   *x = *x + relative_dir[chosen_direction][0];
26   *y = *y + relative_dir[chosen_direction][1];
27   return chosen_direction;
28 }
```

Listing 9: The `direction_choice()` function

For a better understanding of how this function works, we will apply it on an example, illustrated in Figure 5. Suppose we have currently arrived with our DFS in the cell $(3, 5)$ which is situated on the right edge of a 7×7 maze. Cell $(3, 3)$ in the left is visited (we came here from that cell), therefore our function has to return a random direction between *down* and *up*, namely the number 1 or the number 3.

It firstly checks using the repetitive structure `for` what are the valid directions. The two conditions for a direction to be true (lines 6 and 7 of Listing 9) are that the new cell lies inside the maze and is not visited. The logical operator being `and`, if the first condition is not passed the second one is not even checked anymore so we will not get any error related to checking out-of-bounds indices of the matrix `visited`.

If the direction is claimed to be valid, the variable `option_number`, which stores the total number of valid directions, is incremented. However, for later, we will need to know specifically what are those valid directions, not only how many they are. That is why we defined another variable,

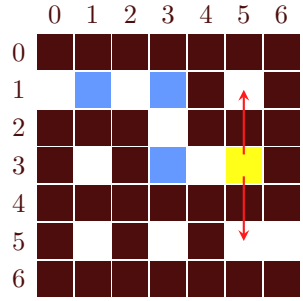


Figure 5: Example of choosing a direction

`option_mask`, which is a binary number (bitmask) between $0000_{(2)}$ and $1111_{(2)}$. This number has a 1 on the position corresponding to a valid direction and a 0 on the position corresponding to an invalid direction.

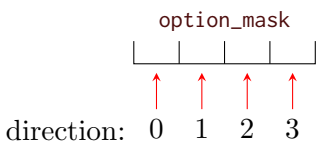


Figure 6: The construction of the `option_mask` number

Note how this number is constructed, by multiplying it by 2 at each step (line 5 of the Listing 9). Therefore, the most significant (leftmost) bit corresponds to direction 0, the next one to direction 1 and so on. For our example, since the valid directions are 1 and 3, the `option_mask` variable equals $0101_{(2)}$. Of course, before continuing, our function returns -1 if no valid direction was found.

Now the problem of choosing a valid direction is reduced to choosing randomly a bit of 1 from `option_mask`. We have `option_number` bits of 1 in `option_mask`. Our result is the `choice`th bit of 1 from right to left. The variable `choice` is initialized with a random number between 0 and `option_number` -1 and incremented afterwards, because we prefer labelling the bits of 1 beginning with 1, not with 0. The repetitive structure `while` analyzes the rightmost bit from the bitmask. At each step, it destroys the bitmask from the right until it reaches the `choice`th bit of 1 in it. The variable `option_counter` stores how many bits of 1 we have encountered up to the present (at the beginning, 0). At each step, this variable is compared with `choice`. When they are equal, it means that we have arrived to the desired bit and the search can be stopped. If not, we check whether the bit we have arrived to is a 1 (line 19 of the Listing 9). If so, we increment the variable `option_counter` which tells us the number of bits of 1 encountered up to this point. The variable `chosen_direction` represents the direction that corresponds to the position of the bit we are currently analyzing (see Figure 5). Taking into consideration that the analization of the bitmask begins with the least significant (rightmost) bit, that bit corresponds with direction 3, as it can be observed in the figure, hence `chosen_direction` is initialized with 4 and decremented at each step (line 22 of Listing 9). Moreover, at each step, the bitmask is divided by 2, all bits being shifted to the right, therefore analyzing the next bit to the left.

When the `while` loop is exited, we have arrived at the desired direction. The coordinates (`xnew`, `ynew`) of the new cell is updated and the function returns the direction it has chosen.

For our example, `option_number` is 2, we have 2 valid directions, therefore `choice` will be assigned

a value (0 or 1) returned by `random_dir()`. Suppose that value is 1. It is then incremented to 2. It means that, the `while` loop will have to find on what position the **second** bit of 1 is situated (counting from right to left). Each iteration of the loop is described in Table 1.

Iteration	Variables' values modifications
condition	`option_counter` (which is 0) is checked against `choice` (2). The test passes, hence we enter in the loop.
1	`option_mask` is $0101_{(2)}$. Its rightmost bit is compared to 1. The condition is satisfied, hence `option_counter` becomes 1. `chosen_direction` becomes 3. `option_mask` becomes $010_{(2)}$.
condition	`option_counter` (1) is checked against `choice` (2). The test passes, hence we enter in the loop once more.
2	The rightmost bit of `option_mask` is compared to 1. The test fails; nothing happens. `chosen_direction` becomes 2. `option_mask` becomes $01_{(2)}$.
condition	`option_counter` (1) is checked against `choice` (2). The test passes, hence we enter in the loop once more.
3	The rightmost bit of `option_mask` is compared to 1. The test passes, hence `option_counter` becomes 2. `chosen_direction` becomes 1. `option_mask` becomes $0_{(2)}$.
condition	`option_counter` (2) is checked against `choice` (2). The test fails, hence we exit the loop.

Table 1: The iterations of the `while` loop of the `direction_choice()` function

The result after the loop is stored in `chosen_direction`, which is 1, hence our function has chosen to go down, into cell $(5, 5)$, which is a valid option.

2.2.7 The DFS function

We implemented the DFS function iteratively, for an easier understanding of the source code. Listing 10 presents the `iterative_dfs()` function, which receives as parameters the coordinates (`xstart`, `ystart`) of the cell we begin with, the dimension of the maze in the form `xmax`, `ymax`, the `maze` matrix and the `visited` matrix.

```
void iterative_dfs(int xstart, int ystart, int xmax, int ymax, int **maze, int **visited) {
  cell *top = NULL;
  int curr_x = xstart, curr_y = ystart, old_x = xstart, old_y = ystart, direction;

  do {
    direction = direction_choice(&curr_x, &curr_y, xmax, ymax, visited);
    if(direction != -1) {
      *(*(visited + curr_x) + curr_y) = 1;
      top = stack_add(top, old_x, old_y);
      smash_walls(curr_x, curr_y, direction, maze);
    }
    else {
      if(!is_empty(top)) {
```

```
14          curr_x = top->x;
15          curr_y = top->y;
16          top = rem(top);
17        }
18      }
19      old_x = curr_x;
20      old_y = curr_y;
21    } while(!is_empty(top));
22  }
```

Listing 10: The `iterative_dfs()` function

First we define and initialize the top of the stack and the coordinates of the current an previous cells (at the beginning they all equal the coordinates of the starting cell, but they will modify during the DFS run). Then, the algorithm runs until the stack empties completely (line 21 of Listing 10). We preferred using the repetitive structure `do ... while` instead of `while` because at the very beginning the stack is empty and if we first tested the condition, it would never enter the loop. Inside the loop, we choose a random direction to go to from the current cell (line 6). If there is a valid direction (lines 7 - 11), we mark the current cell as visited, add the previous cell to the top of the stack and break the wall between the previous and the current cell. If no direction is available to go to, we move to the cell which is at the top of the stack (lines 14 - 15) and remove that cell from the stack (line 16). However, we have to additionally check whether the stack is empty prior to this, because in the trivial case of a 3×3 maze, not even the first cell will be added to the stack since there is nowhere to go to in a 3×3 grid, and therefore, in this case, the algorithm will reach line 13 with the top of the stack being `NULL`. Finally, we update the coordinates of the previous cell with the coordinates of the current one, preparing for the next execution of the loop.

We have also implemented a recursive form of the DFS function, presented in Listing 11. The main benefit of this function is that we do not need a stack anymore, because the local memory of the function is the stack itself; with every call of the function, a new element is added onto the stack, containing information about the previous cell (exactly what is needed for a DFS) in the form of the local variable `direction`. Remark the simplicity of the code for this function.

```
void recursive_dfs(int x, int y, int xmax, int ymax, int **maze, int **visited) {
  int direction;
  while((direction = direction_choice(&x, &y, xmax, ymax, visited)) != -1) {
    *(*(visited + x) + y) = 1;
    smash_walls(x, y, direction, maze);
    recursive_dfs(x, y, xmax, ymax, maze, visited);
  }
}
```

Listing 11: The `recursive_dfs()` function

Any of the two functions can be used and they work perfectly. What is more surprising, is that they both generate exactly the same mazes for the same given seed. It is because the two functions build the maze in the same manner and the order of decisions they take concerning what direction to go to is the same for both functions.

2.2.8 The main function

Now we are ready to write the main function that will take all the necessary steps to generate the maze. The function `generate_maze()` (Listing 12) requires as parameters the `width` and the `height` of the maze we want to obtain, and also the seed based on which the maze is generated.

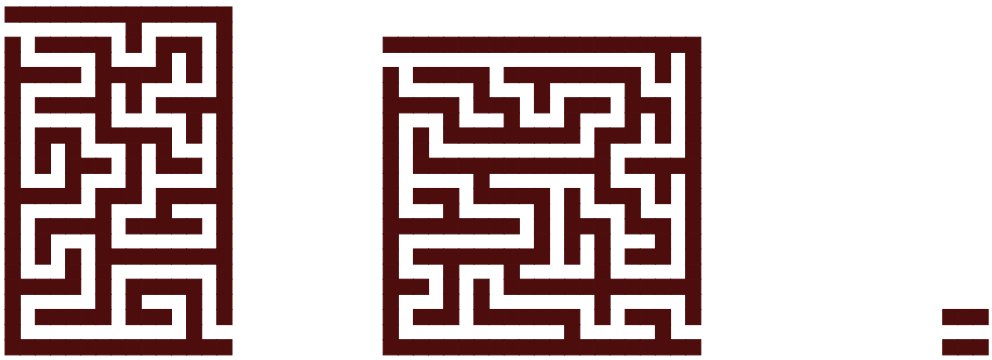
It returns a pointer to the `maze` matrix. If the user accidentally inputs even dimensions, the program will not crash, but will instead modify them to the odd number above them by adding a 1 (lines 2 and 3 of Listing 12).

After generating many mazes using `generate_maze()` we observed that starting from cell $(1, 1)$, the maze has no ramification for a while, and the reason is clear: the DFS does not make ramifications until it gets stuck and because at the beginning there are very few visited cells it does not get stuck. For this reason, the maze may seem easy or boring because you do not have to take many decisions. The solution that we came up with is the DFS starting not from $(1, 1)$ but, instead, from (`height - 2`, `width - 2`), which is the finish cell. In line 10 of Listing 12 the starting cell is marked as visited and after that the `iterative_dfs()` or the `recursive_dfs()` function is called, starting from the cell we agreed upon.

```
1  int **generate_maze(int *width, int *height, int seed) {
2    (*width) += ((*width) % 2 == 0) ? 1 : 0;
3    (*height) += ((*height) % 2 == 0) ? 1 : 0;
4
5    srand(seed);
6
7    int **visited = initialize_visited(*width, *height);
8    int **maze = initialize_maze(*width, *height);
9
10   *(*(visited + *height - 2) + *width - 2) = 1;
11   iterative_dfs(*height - 2, *width - 2, *height, *width, maze, visited);
12   /*
13   recursive_dfs(*height - 2, *width - 2, *height, *width, maze, visited);
14   */
15   clear_mem(*height, &visited);
16   return maze;
17 }
```

Listing 12: The `generate_maze()` function

Figure 7 presents examples of mazes generated by our function. Can you solve them? Then try to solve the mazes from the finish cell to the start cell (in the same way as the DFS) and notice that it is much easier and you barely have to take any decision. That is why we preferred the finish cell as the starting point for the DFS.



a) `width` $= 15$, `height` $= 23$, `seed` $= 2024$

b) `width` $= 21$, `height` $= 21$, `seed` $= 256$

c) `width` $= 3$, `height` $= 3$, `seed` $= 71$

Figure 7: Examples of mazes generated by `generate_maze()`

3 Maze solving

In our project, we preferred that the robot, which is placed in the starting cell, already knows the configuration of the maze, so instead of discovering it by walking on every corridor, which would take an impressive amount of time, it will follow only the necessary steps to arrive at the finish cell. Hovever, to do that, it needs an algorithm that outputs the path to the exit. For diversifying our project, we used the breadth-first-search (BFS) algorithm.

3.1 Theoretical considerations

Applied on a maze, the BFS is easy to understand. First, it goes into the starting cell. Then it visits all the neighbouring cells of the starting cell. Then it visits the neighbouring cells of those cells, and so on, until it reaches the finish cell. In other words, we can say that the BFS expands like a fire, going radially outwards from a cell to the neighbouring cells. This behaviour is illustrated in Figure 8, where, for the ease of understanding, the cells are labelled in the order they are visited (instead of their coordinates) and coloured according to how early they were visited.

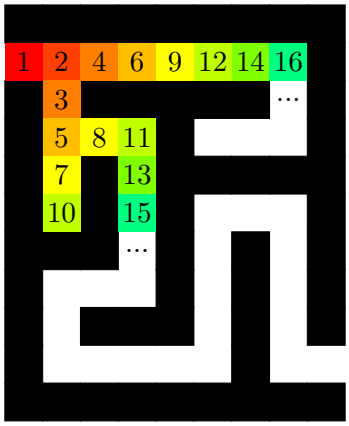


Figure 8: The behaviour of the BFS algorithm on a maze

Remark how the expansion occurs: at each step we visit some cells; suppose we are now at the step coloured in light orange in the Figure 8, when we visit cell 5 and then we visit cell 6. At the next step, we first come back and check for the neighbours of cell 5 (which are 7 and 8) and visit them, and then return to cell 6 and visit its neighbour 9. At this step, the order of visiting the cells is: 7, then 8, then 9. At the next step, we again come back to the first cell visited in the previous step (which is 7) and visit its neighbour 10, then go to cell 8 and visit its neighbour 11 and lastly go to cell 9 and visit its neighbour 12. It is very important that, at each step, we first check the neighbours of the **first** cell visited in the previous step, not the ones of the **last** cell. Otherwise, this algorithm will become a DFS, expanding only on one branch until it gets stuck on it and leaving the other branches behind, while the idea of BFS is to expand simultaneously in all branches.

As we have previously seen, we need to memorize all the cells that have been visited in the previous step and we have to be able to always access the **first** cell that was memorized, on the principle *first-in-first-out* (FIFO). A data structure working on this principle is called a *queue*. You can imagine it as a double-ended box, in which we can insert objects only through the bottom opening and we can take out objects only through the top opening.

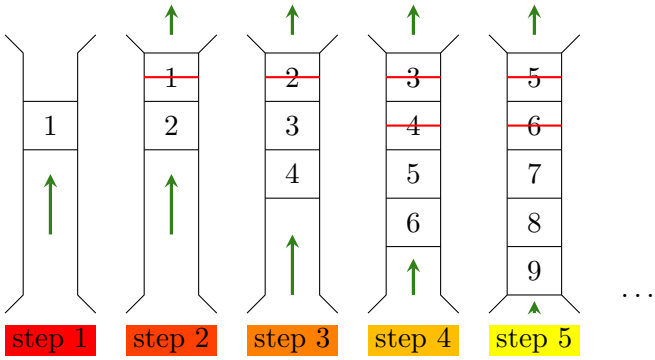


Figure 9: The BFS implementation with queue

Figure 9 presents the behaviour of the queue while running the BFS algorithm

- Step 1: We are in cell 1, we add it to the queue through the bottom.
- Step 2: We look at the top of the queue and remove that cell. It is cell 1, we visit and add its neighbour 2 at the bottom of the queue.
- Step 3: We look at the top of the queue and remove the first cell, cell 2. We visit its neighbours 3 and 4 and add them one by one at the bottom of the queue.
- Step 4: We look at the top of the queue and remove cell 3, visit its neighbour 5 and immediately add it at the bottom of the queue. We remove cell 4 from the top of the queue, visit its neighbour 6 and immediately add it at the bottom of the queue.

The process continues until we reach the destination cell, or, for an easier implementation, until the queue empties completely. It is clear that when all the branches terminate or we reach the finish cell, we only remove the top cells from the queue and we have nothing to add at the bottom, thus emptying the queue.

Remark that we still have to remember the cells that have previously been visited (as for the DFS), because, for example, when reaching cell 4, cell 2 is not a valid cell to go to because if doing so, we would return to the origin and, moreover, enter an infinite loop.

3.2 Implementation

3.2.1 The queue

The necessary functions for manipulating a queue were also defined in the *structures.c* file. Removing an element from a queue is equivalent with removing an element from a stack so we will use the function `rem()` for both structures. The function needed for adding an element to a queue is presented in Listing 13. The function for checking whether the stack is empty, `is_empty()`, works fine with the queue too.

```
cell *queue_add(cell *old_cell, int cell_x, int cell_y) {
  cell *new_cell;
  new_cell = (cell *)malloc(sizeof(cell));
  new_cell->x = cell_x;
  new_cell->y = cell_y;
  new_cell->link = NULL;
  if(old_cell != NULL) {
```

```
8     old_cell->link = new_cell;
9   }
10  return new_cell;
11 }
```

Listing 13: The `queue_add()` function

For a queue, we will need to memorize both the top and the bottom cells, because we add into the bottom and remove from top. Instead of top and bottom, they will be called `front` and `back`. It is very important to always initialize the `link` member of the `back` cell with `NULL` (line 6 of Listing 13), because when we reach the end of the algorithm, it removes even the last cell from the queue, which is the `back`, therefore `front` must then equal `NULL`, in order for function `is_empty()` to correctly interpret that the queue emptied.

3.2.2 The parent matrix

All the functions related to solving the maze were also defined in the file *maze.c*.

A more challenging task is, when the algorithm has reached the finish cell, to be able to tell what is the path taken to reach that cell. The solution to this problem is intuitive. We will take another matrix, in which, for each cell, we will memorize from what direction the algorithm came into that cell. This matrix is called `parent` and Listing 14 presents the function that initializes it. First, all the cells have the direction -1 and the cell from which we start has -2, we will immediately see why. Furthermore, we can use this matrix also for checking whether a cell has or has not been visited yet. To a cell that has been visited is assigned a value between 0 and 3 in this matrix, or has the value -2 (the starting cell). Therefore, to a not-visited cell corresponds value -1 in this matrix.

```
int **initialize_parent(int width, int height, int xstart, int ystart) {
  int **parent = (int **)calloc(sizeof(int *), height);
  for(int row = 0; row < height; row++) {
    *(parent + row) = (int *)calloc(sizeof(int), width);
    for(int column = 0; column < width; column++) {
      *(*(parent + row) + column) = -1;
    }
  }
  *(*(parent + xstart) + ystart) = -2;
  return parent;
}
```

Listing 14: The `initialize_parent()` function

3.2.3 Other preparatory functions

In order to see whether going into another cell is a valid option we have to test multiple things: if the new cell is inside the maze, if it is not a wall and if it has not been visited yet. For easing the source code, we grouped all these conditions in a separate function called `is_valid_direction()` which, based on the information provided by the `maze` and `parent` matrices, returns 1 if the new cell is valid or 0 if contrary.

```
int is_valid_direction(int x, int y, int width, int height, int **maze, int **parent) {
  return (in_matrix(x, y, height, width) &&
          (*(*(maze + x) + y) == 0) &&
          (*(*(parent + x) + y) == -1));
```

```
}
```

Listing 15: The `is_valid_direction()` function

3.2.4 The BFS function

The BFS function is an iterative function presented in Listing 16.

```
void bfs(int xstart, int ystart, int width, int height, int **maze, int **parent) {
  cell *front = queue_add(NULL, xstart, ystart);
  cell *back = front;

  while(!is_empty(front)) {
    int curr_x = front->x;
    int curr_y = front->y;

    //we test all the four directions
    for(int direction = 0; direction < 4; direction++) {
      int new_x = curr_x + relative_dir[direction][0] / 2;
      int new_y = curr_y + relative_dir[direction][1] / 2;
      if(is_valid_direction(new_x, new_y, width, height, maze, parent)) {
        *(*(parent + new_x) + new_y) = direction;
        back = queue_add(back, new_x, new_y);
      }
    }
    front = rem(front);
  }
}
```

Listing 16: The `bfs()` function

The function takes as parameters the coordinates of the starting cell, (`xstart`, `ystart`), the dimensions of the maze and the `maze` and `parent` matrices.

Firstly it adds the starting cell to the queue. At the beginning, `back` equals `front`, because there is a single cell inside the queue. Then the algorithm enters the `while` loop which runs until the queue empties. The cell at the front of the queue is analyzed as current cell of coordinates (`curr_x`, `curr_y`). Then it tries to go into all 4 directions. If possible, we mark in the `parent` matrix the direction we took to get to the new cell and the new cell is added at the back of the queue. Finally, the current cell is removed from the queue.

3.2.5 Recovering the path

For recovering the path that was taken in order to reach the destination cell, the `parent` matrix is used. The idea is to begin with the finish cell and, from that cell, go where the `parent` tells you to go until you reach the starting cell, however keeping in mind that the directions are reversed. For an arbitrary cell (i, j), if for instance `parent[i][j]` $= 2$ it means that we went leftwards to get into this cell, so now we need to go rightwards in order to reach the starting cell.

Figure 10 presents the `parent` matrix for an arbitrary maze. You can observe that the cells with walls remain unvisited (-1), while all the other cells apart from the start one tell us what direction was taken to get there. For instance, the finish cell has 0 meaning that the algorithm went rightwards to get there (obviously, it came from the cell to its left).

Here is why we needed to mark the starting cell with -2 instead of -1. If it was marked like an ordinary unvisited cell, with -1, when the algorithm reached the second cell (cell $(1, 1)$) it would evaluate the starting cell as an unvisited cell and go again in it. This is not a big problem,

-1	-1	-1	-1	-1	-1	-1	-1	-1
-2	0	0	0	0	0	0	0	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	1	0	0	-1	2	2	1	-1
-1	1	-1	1	-1	-1	-1	-1	-1
-1	1	-1	1	-1	3	0	0	-1
-1	-1	-1	1	-1	3	-1	1	-1
-1	2	2	1	-1	3	-1	1	-1
-1	1	-1	-1	-1	3	-1	1	-1
-1	1	0	0	0	0	-1	1	0
-1	-1	-1	-1	-1	-1	-1	-1	-1

Figure 10: The `parent` matrix for an arbitrary maze

because it would stop here, having nowhere to go to (cell $(1,1)$ being marked as visited). The problem is that the `parent` of the starting cell would be assigned the value 2, thus indicating that the BFS came here from cell $(1,1)$. On the other hand, the `parent` of $(1,1)$ indicates that the algorithm went there from cell $(1,0)$, which was marked at the first step. Therefore, if we wanted to recover the path in this case, when we arrived at cell $(1,0)$, the starting cell, the algoritm would jump from $(1,0)$ to $(1,1)$ and enter an infinite loop. That is why, we will see a bit later, the stopping condition of the loop that recovers the path is when it reaches a cell with `parent` equal to -2.

One more step is needed before finally writing the function that recovers the path. The directions inscribed in the `parent` matrix are absolute directions. For our application however, the robot needs to know when to turn left, when to turn right and when to go forward. We want to obtain therefore an array of "commands" for the robot, more precisely an array called `path`, where

$$\texttt{path[i]} = \begin{cases} 0, \text{ if the robot has to go forward} \\ 1, \text{ if the robot has to turn right} \\ 2, \text{ if the robot has to turn left} \end{cases} \tag{2}$$

in the ith cell it crosses on its way to the finish cell.

The function `direction_relativization()` returns the integer corresponding to the move the robot has to make, being given as parameters the direction the robot took to get into the current cell, called `current_dir`, and the direction the robot will take to get into the next cell, called `next_dir`. Listing 17 presents this function.

```
int direction_relativization(int current_dir, int next_dir) {
  if(current_dir == next_dir) { //forward
    return 0;
  }
  if((current_dir + 4 - next_dir) % 4 == 1) { //turn left
    return 2;
  }
  //turn right
```

```
  return 1;
}
```

Listing 17: The `direction_relativization()` function

Now we are finally able to write the function that recovers the path. Listing 18 presents the function `recover_path()`.

```
void recover_path(int xstop, int ystop, int **parent, char *file_name) {
  cell *top = NULL;
  int curr_x = xstop, curr_y = ystop;
  int next_direction = 0, actual_direction, previous_direction = 0, previous_rel_dir = 0;
  int length = 2;
  while(*(*(parent + curr_x) + curr_y) != -2) {
    top = stack_add(top, *(*(parent + curr_x) + curr_y), 0);
    actual_direction = (*(*(parent + curr_x) + curr_y) + 2) % 4;
    curr_x = curr_x + relative_dir[actual_direction][0] / 2;
    curr_y = curr_y + relative_dir[actual_direction][1] / 2;
    if(previous_direction == actual_direction)
      length++;
    previous_direction = actual_direction;
  }

  FILE *path_file = fopen(file_name, "w");
  fprintf(path_file, "%d\n", length);
  while(!is_empty(top)) {
    actual_direction = top->x;
    if(actual_direction == -2) {
      actual_direction = 0;
    }
    next_direction = (top->link != NULL) ? top->link->x : 0;

    if(previous_rel_dir == 0) {
      fprintf(path_file, "%d %d %d\n", direction_relativization(actual_direction, next_direction),
     curr_x, curr_y);
    }
    curr_x = curr_x + relative_dir[actual_direction][0] / 2;
    curr_y = curr_y + relative_dir[actual_direction][1] / 2;
    top = rem(top);
    previous_rel_dir = direction_relativization(actual_direction, next_direction);
  }
  fprintf(path_file, "%d %d %d\n", -1, curr_x, curr_y);
  fclose(path_file);
}
```

Listing 18: The `recover_path()` function

The function takes as parameters the coordinates of the finish cell, from where it starts the recovering, the pointer to the `parent` matrix and the name of the file it will print the solution to. The function prints the solution into a file and does not return the path as a pointer to an array for example (a more elegant solution), because the robot will have to access it and there is no other possible way of transmitting it because the code that commands the robot is a completely different executable.

Notice that we recover the steps from the finish cell to the starting cell, but the commands for the robot must be given from the starting cell. That is why, we will first place all the commands in a stack, and then remove them one by one in reverse order and add them to our array. The command is only an integer, our stack being implemented with two integer members, that is why we will use its `x` member to memorize the command and leave the member `y` as 0 always.

The `recover_path()` function prints not only the `path` array into the file, but also the absolute coordinates of the cells the robot has to go to. It will turn out to be very useful later for the robot, to decide whether he has arrived in the required cell, using his GPS device.

3.2.6 The main function

Finally, we can write the main public function that the robot is able to call for finding the way out from the maze.

```
void solve_maze(int width, int height, int **maze, int xstart, int ystart, int xstop, int ystop, char
    *file_name) {
  int **parent = initialize_parent(width, height, xstart, ystart);
  bfs(xstart, ystart, width, height, maze, parent);
  recover_path(xstop, ystop, parent, file_name);
  clear_mem(height, &parent);
}
```

Listing 19: The `solve_maze()` function

4 Simulation

4.1 Creating the environment

The first step is to install webots. This is easily done by visiting https://cyberbotics.com/ and clicking the big green button captioned *Download*.

Next, we need to know how to use this simulation environment, so we carefully read the documentation, and especially the tutorials found at https://cyberbotics.com/doc/guide/tutorials. Because the reader might not be familiar with webots, we will now give a brief overview that will ease the understanding of our project.

The components and the characteristics of the simulation are organised into nodes. Each node has one or more fields and can inherit other nodes. For example, our robot is a node. The floor is a node, the background is, the camera information is. Without the robot, webots is like a 3D physics simulator. Any robot in webots has a controller, where lies all the code that determines the behaviour of the robot.

After following all the turorials mentioned above we begin working on the project, which can be split into two parts: the labyrinth and the robot.

The labyrinth will consist of many cubes stuck to each other, which means the height will equal the width of the wall. One such cube can be manually inserted into the world by adding a new Solid node and modifying its parameters, but we want to have our maze spawned automatically. To accomplish this task we make use of the supervisor, which is a robot that has a supervisor controller and can modify almost everything inside our simulation, like adding nodes, which is what we want.

Each cube's characteristics can be found by manually creating the object and then opening the webots world as a text file. From there we simply copy the description of the cube, which we save inside a .proto file. The only filed that needs to be modified for each cube is the translation field. The supervisor will use this proto file to spawn each cube. The source code of the supervisor controller, saved in the *supervisor.c* file, can be found below.

```
#include <webots/robot.h>
#include <webots/supervisor.h>
#include <stdio.h>
```

```c
#include <stdlib.h>
#include "maze.h"
#include "structures.h"
#define TIME_STEP 32
#define CELL_WIDTH 0.2
```

We included the necessary libraries to operate with functions from the webots API and two user defined libraries which allow us to use the functions described in the previous chapter. The `TIME_STEP` constant tells us how often the controller will cycle: every 32 milliseconds. The `CELL_WIDTH` constant is the universal width of a cell in the simulation.

Next, we present the `spawn_maze()` function, which is key to this controller

```c
void spawn_maze(int **matrix, int rows, int columns, WbFieldRef children_field)
{
  for (int i = 0; i < rows; i++)
    for (int j = 0; j < columns; j++) {
      if (matrix[i][j] == 1) {
        sprintf(position_string, "wall_proto {translation %f %f %f}", CELL_WIDTH * j, CELL_WIDTH *(-1) * i, CELL_WIDTH * 0.5);
        wb_supervisor_field_import_mf_node_from_string(children_field, -1, position_string);
      }
    }
}
```

We map the maze matrix to the simulation: for every 1 in the matrix there will be a box in the maze. There is a function that adds nodes, we only need to specify the place (the children field of our root node) and the position inside the simulation (calculated and stored inside a string). When seen from above, the freshly generated maze looks like the matrix it was generated from (it has the same orientation). For a better understanding of how to add nodes inside the simulation using webots, we encourage the reader to look up the documentation.

After declaring the global variables the `main()` function starts. First, we get handles for the root node and it is children field which will be used to insert the maze.

```c
int main(int argc, char **argv) {

  wb_robot_init();

  WbNodeRef root_node = wb_supervisor_node_get_root();
  WbFieldRef children_field = wb_supervisor_node_get_field(root_node, "children");
```

The dimensions of the maze, as well as the seed, must be provided by the user into the `input.txt` file which must be saved in the same folder as the robot controller. For safety measures, we initialize the `width` and `height` variables with nonzero values. This ensures that in case the file is empty, the controller does not crash.

```c
  int width = 3, height = 3, seed = 23;

  FILE *fp;
  fp = fopen("input.txt", "r");
  if (fp == NULL) {
      printf("Error opening the file.\n");
      return 1;
  }
  fscanf(fp, "%d %d %d", &width, &height, &seed);
  fclose(fp);
```

We spawn the floor in the same manner in which we spawned each box for the walls, but this time we also specify the size to be equal with that of the maze. After that we call the function

to spawn the maze. The process can be slow, especially for big mazes (it takes a few seconds, clock time), but it is not a problem, as the simulation has yet to start ticking.

```
  int **maze = generate_maze(&width, &height, seed);
  solve_maze(width, height, maze, 1, 0, height - 2, width - 1, "path.txt");

  sprintf(floor_position_string, "Basic_floor {translation %f %f 0, size %f %f}", 0.5 * CELL_WIDTH *
     (-1 + width), 0.5 * CELL_WIDTH * (1 - height), CELL_WIDTH * width, CELL_WIDTH * height);
  wb_supervisor_field_import_mf_node_from_string(children_field, 2, floor_position_string);

  spawn_maze(maze, height, width, children_field);

  while (wb_robot_step(TIME_STEP) != -1);
  wb_robot_cleanup();
  return 0;
}
```

The main loop is empty, as all supervisor tasks are done before the simulation starts.

The Worldinfo node needs to have a few fields adjusted. The *basicTimeStep* controlls how often the simulation will refresh, the number set there representing the number of milliseconds that will pass between each step of the simulation. The lower this number is, the more accurate things will be, but also more expensive to compute. A good balance seems to be at 16 ms. Because the maze consists of many cubes that constantly "touch" each other we set the *physicsDisableTime* to 0, which means that any static object will have it's physics disabled after 0 seconds of being idle: for us, this means the walls do not slow down the simulation (unless, of course, the robot will run into them).

Figure 11 depicts an example of a maze generated by the supervisor.

Figure 11: The maze generated by the supervisor for `width` = 15, `height` = 15, `seed` = 13

4.2 The robot

4.2.1 Design

The main idea behind the design of the robot is simplicity. A simple robot is more robust, it can be programmed easier and it is less stressing for the CPU. Our robot needs to fulfill two tasks: to go forward and to turn right or left. A pair of wheels is enough to make the robot go

forward and to turn, because we use differential steering. The body of the robot would serve to stabilize it and to store the necessary components (like battery and motors).

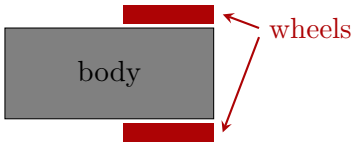


Figure 12: A graphical representation of our robot (top view)

Figure 13 shows an image of the robot from Webots.

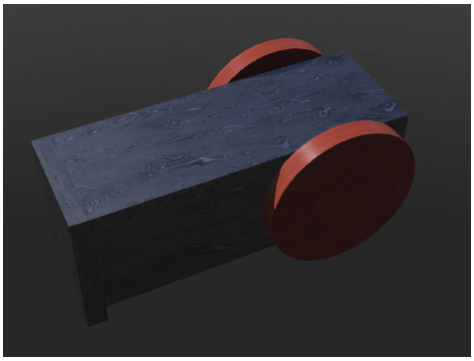


Figure 13: An image of our robot

Figure 14 presents a low level representation of the nodes composing the robot.

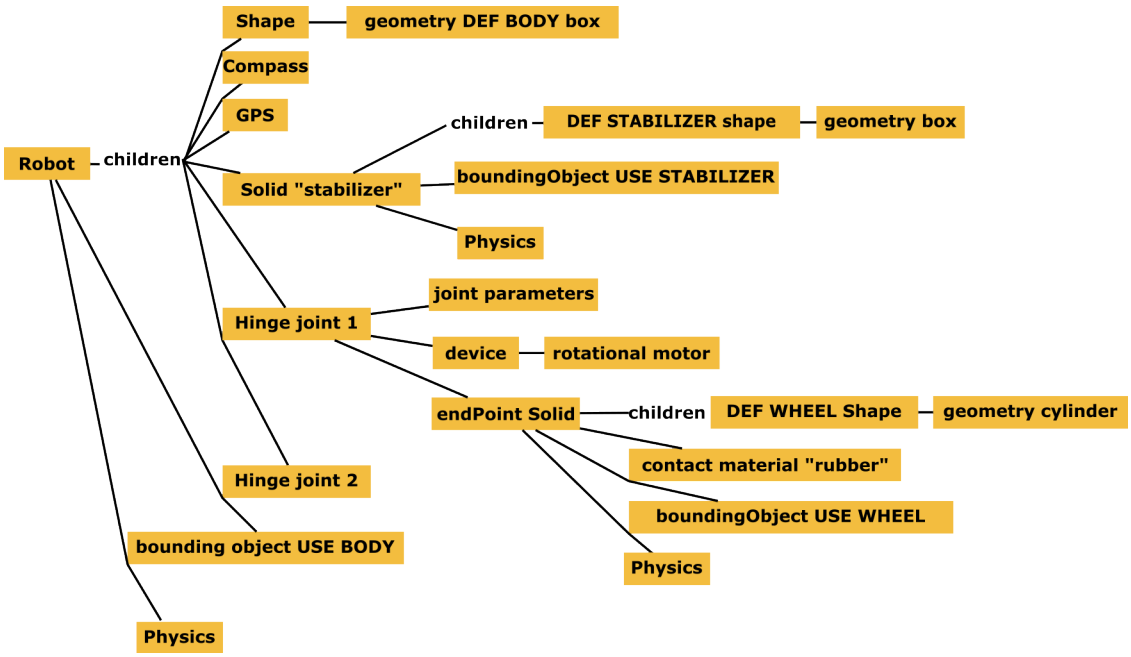


Figure 14: The diagram of the robot

The sizes of the robot were chosen according to the size of our walls: it is wide and long enough to fit confortably inside the maze, but not so small that the challange to solve the maze

becomes trivial. The height does not matter much, as long as the center of gravity is low enough to keep the robot from tilting when steering. Also, it would be interesting to notice the size of the wheels: thinner wheels are more precise (when talking about differential steering), but they have a smaller contact area with the ground, and thus they are more prone to sliding and glitching. The wheels were tuned to be heavier (their density is ten times that of the rest of the robot) in order to negate some of the disatvantages of inertia when turning. We also defined a "rubber" material for them that has higher friction with the floor. Also, we placed a stabilizer that keeps the robot level at all times and has no friction with the ground, which could simulate a universal wheel in real life. The reason why we did not add one in our simulation is because it would unnecessarily slow it down, and the same effect is obtained by using the above mentioned implementation.

4.2.2 Control

The aim for our robot is to get out of the maze decently fast. This is why we give it the directions to follow prior to entering the labyrinth. So the robot only knows two things so far: a sequence of relative directions (go forward, turn left or turn right) and a sequence of coordinates corresponding to the route it has to follow inside the maze. The tools at its disposal are a GPS and a compass.

All the commands necessary for the robot to operate have been defined in the controller file called *robot_guide.c*. The way the controller works is the following: for each cell of the maze, check the direction the robot needs to go and set the wheel velocities accordingly. Now there are two options: if the robot moves forward, then keep it going until it reaches a checkpoint position, or, if the robot has to turn, keep it turning until its orientation changes by 90°. Then, update the direction and repeat. Below we present the implementation.

```c
#include <webots/motor.h>
#include <webots/robot.h>
#include <webots/gps.h>
#include <webots/compass.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "maze.h"
#include "structures.h"

#define TIME_STEP 16
#define SPEED 5
#define RATIO 1.531
#define CELL_WIDTH 0.2

const double velocities[3][2] = {{1, 1}, {1, RATIO}, {RATIO, 1}};
```

After we include the necessary libraries we define some constants of which we mention

1. `RATIO`, which represents the ratio of the angular speeds of the two wheels while turning. Since both wheels have to perform the circular path in the same time, `RATIO` also equals the ratio of the lengths of the paths the two wheels follow while taking a turn. The calculation logic of this constant was visually represented in Figure 15. If we denote by W half the width of the walls, by d_1 the distance from the outer wheel to the wall about which the rotation is performed and by d_2 the distance from the inner wheel to the same wall, it

becomes clear that

$$\texttt{RATIO} = \frac{d_1 + W}{d_2 + W}$$

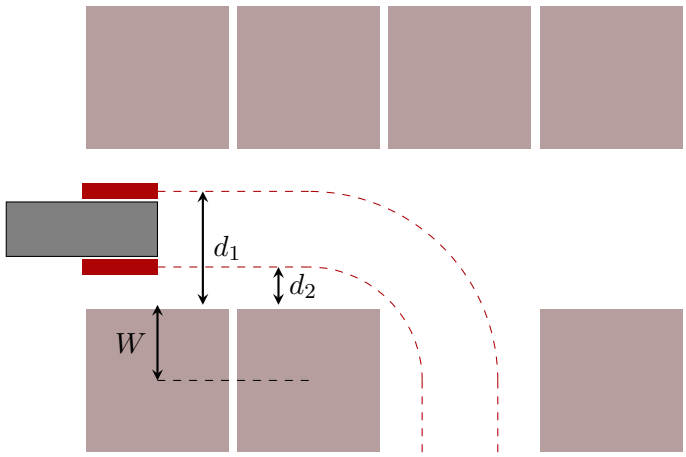


Figure 15: The ratio of the speeds of the wheels while turning

2. `SPEED`, which represents the magnitude of the angular speed of the wheels while the robot goes forward. It was set to 5 radians / second. The higher the speed, the bigger the probability of error, and we found empirically that around 5 we can expect reliable behaviour from the robot even for big matrices. A higher speed would need to be compensated by a lower time step for the simulation (which means a heavier load for the CPU).

3. `CELL_WIDTH` equals the width of a cell in the simulation.

Next, we present the function that obtains the information given to the robot prior to entering the labyrinth.

```c
int read_path(int **path, double ***absolute_coord) {
  int steps = 0;
  FILE *path_file = fopen("../supervisor/path.txt", "r");
  fscanf(path_file, "%d", &steps);

  (*path) = (int *)calloc(sizeof(int), steps);
  (*absolute_coord) = (double **)calloc(sizeof(double *), steps);
  for(int step = 0; step < steps; step++) {
    *((*absolute_coord) + step) = (double *)calloc(sizeof(double), 2);
    fscanf(path_file, "%d%lf%lf", (*path) + step, *((*absolute_coord) + step) + 1, *((*absolute_coord) + step));
    *(*((*absolute_coord) + step)) = CELL_WIDTH * *(*((*absolute_coord) + step));
    *(*((*absolute_coord) + step) + 1) = (- 1.0) * CELL_WIDTH * *(*((*absolute_coord) + step) + 1);
  }
  fclose(path_file);
  return steps;
}
```

Listing 20: The `read_path()` function

The array `path` stores the directions that need to be followed in each square, while in `absolute_coord` we store the coordinates (relative to 0 inside the simulation) of the centers of each square.

We also use a function to determine the exit condition and we will come back to it when it becomes relevant. It is mandatory for the robot controller to call the `wb_robot_step()` function regularly, otherwise the sensors and actuators will not be updated and the simulator will block. For this reason, The central part of the controller consists of a loop inside of which `wb_robot_step()` is called. The `main()` begins with initialization of variables, establishing a link between webots and the controller and enabling the devices we intend to use.

```
int main(int argc, char **argv) {
  wb_robot_init();

  WbDeviceTag fr_motor = wb_robot_get_device("front_right_motor");
  WbDeviceTag fl_motor = wb_robot_get_device("front_left_motor");
  WbDeviceTag gps = wb_robot_get_device("gps");
  WbDeviceTag compass = wb_robot_get_device("compass");
  wb_gps_enable(gps, TIME_STEP);
  wb_compass_enable(compass, TIME_STEP);

  double left_speed = SPEED, right_speed = SPEED;
  double *position, *orientation;
  int step = 0, correction = 1;

  wb_motor_set_velocity(fr_motor, 0);
  wb_motor_set_velocity(fl_motor, 0);
  wb_motor_set_position(fl_motor, INFINITY);
  wb_motor_set_position(fr_motor, INFINITY);
```

The last 4 lines above are resposible for the initial state of the robot wheels: they are set to move endlessly, but the initial speed is set to 0.

```
  wb_robot_step(TIME_STEP);
  int *path;
  double **absolute_coord;
  int total_steps = read_path(&path, &absolute_coord);
  int req_step, req_orientation, req_sign;

  position = wb_gps_get_values(gps);
  orientation = wb_compass_get_values(compass);
```

We store the values recorded by the GPS and by the compass and we get the path. Now, because our path is generated by the supervisor, we need to add a certain delay between printing into the file and reading from the file. The supervisor writes the information to `path.txt` before the simulation starts and after 16 ms the robot will read this information. We have to give time to the supervisor to write into this file and not risk to read it while still being empty.

Finally, the main loop is presented in Listing 21.

```
    while(path[step] != -1) {
    req_step = path[step];
    req_orientation = (fabs(orientation[0]) > 0.5) ? 1 : 0;
    req_sign = (position[req_orientation] - absolute_coord[step + 1][req_orientation] < 0) ? -1 : 1;
    wb_motor_set_velocity(fr_motor, SPEED * velocities[path[step]][0]);
    wb_motor_set_velocity(fl_motor, SPEED * velocities[path[step]][1]);
    do {
      position = wb_gps_get_values(gps);
      orientation = wb_compass_get_values(compass);
      wb_robot_step(TIME_STEP);
    } while(exit_condition(req_step, req_orientation, req_sign, step, absolute_coord, orientation,
     position));
    step++;
```

```
  }

  wb_motor_set_velocity(fr_motor, 0);
  wb_motor_set_velocity(fl_motor, 0);

  wb_robot_cleanup();
  return 0;
}
```

Listing 21: The main loop of the controller

We will break the explanation in two parts: if the robot goes forward or if it turns. It is very easy to implement movement, the difficulty lies in deciding precisely when to implement each movement.

If it goes forward, then we use the GPS to track its location and to check when the robot needs to change direction. For every square that the robot needs to walk we store its center as the intersection of two lines, one on the x axis and one on the y axis. For every square, the robot will keep going until it crosses the line passing through the center of the target square. When the wheels touch that line the exit condition is met and the robot will do the process again.

If the robot has to turn, then we check the compass. Because of the way the labyrinth is generated, at every turn the robot must change its orientation (it is either oriented along x axis or y axis). The `wb_compass_get_values()` function does not return values that are directly proportional with the rotation of the robot (while the GPS returns exactly the values of the translation field). This is convenient, as we found out that when close to the desired orientation, the compass is extremely precise. Once we reach that desired orientation we get the next direction.

Lines 62 to 65 tell the robot where it has to go, lines 66 and 67 make the robot actually move with the correct velocity for each wheel. Lines 68 to 72 represent the loop that prevent the robot from entering into the next step of the path before arriving to checkpoint. At each simulation step, the position and the orientation are read from the devices mounted on the robot. These values are checked against the values they should have when arriving to the next checkpoint in the function `exit_condition()`, presented in Listing 22.

```
1 int exit_condition(int req_step, int req_orientation, int req_sign, int step, double **absolute_coord
      , double *orientation, double *position) {
2   if(req_step == 0) {
3     return (req_sign * (position[req_orientation] - absolute_coord[step + 1][req_orientation]) >=
      0.001);
4   }
5   return (fabs(orientation[req_orientation]) <= 0.9999);
6 }
```

Listing 22: The `exit_condition()` function

When the robot goes forward, or, equivalently, the current step in the path is 0 (line 2 of Listing 22) the exit condition is related to the absolute position of the robot, stored in the `position` vector, which are compared to the coordinates of the cell it must arrive to, stored in the `absolute_coord` vector of pairs.

When the robot is turning (line 5), the absolute value of the orientation is checked. Observe that in the case of the orientation, the precision is 10 times higher. This was set according to the performance characteristics of the two devices: the GPS and the compass.

Finally, once the robot has reached the destination it will stop and the controller will exit.

Appendices

A - *Source and header files*

maze.h

```c
#ifndef MAZE_H_INCLUDED
#define MAZE_H_INCLUDED

#include "structures.h"

static int random_dir(int max_dir);
static int in_matrix(int x, int y, int xmax, int ymax);
static int direction_choice(int *x, int *y, int xmax, int ymax, int **visited);
static int **initialize_visited(int width, int height);
static int **initialize_parent(int width, int height, int xstart, int ystart);
static void smash_walls(int x, int y, int direction, int **maze);
static void iterative_dfs(int x, int y, int xmax, int ymax, int **maze, int **visited);
static void recursive_dfs(int x, int y, int xmax, int ymax, int **maze, int **visited);
static int is_valid_direction(int x, int y, int width, int height, int **maze, int **parent);
static void bfs(int xstart, int ystart, int width, int height, int **maze, int **parent);
extern void clear_mem(int height, int ***matrix);
static int direction_relativization(int current_dir, int next_dir);
static void recover_path(int xstop, int ystop, int **parent, char *file_name);
extern void print_maze(int width, int height, int **maze);
extern int **generate_maze(int *width, int *height, int seed);
extern void solve_maze(int width, int height, int **maze, int xstart, int ystart, int xstop, int
    ystop, char *file_name);

#endif
```

maze.c

```c
#include <stdlib.h>
#include <stdio.h>
#include "maze.h"
#include "structures.h"

static int relative_dir[4][2] = {{0,  2}, {2, 0}, {0, -2}, {-2, 0}};

int random_dir(int max_dir) {
  return rand() % max_dir;
}

int in_matrix(int x, int y, int xmax, int ymax) {
  return (x >= 0) && (x < xmax) && (y >= 0) && (y < ymax);
}

int direction_choice(int *x, int *y, int xmax, int ymax, int **visited) {
  int option_mask = 0, option_number = 0, choice = 0, option_counter = 0, chosen_direction = 4;

  for(int direction = 0; direction < 4; direction++) {
    option_mask *= 2;
    if(in_matrix(*x + relative_dir[direction][0], *y + relative_dir[direction][1], xmax, ymax) &&
       !(*(*(visited + *x + relative_dir[direction][0]) + *y + relative_dir[direction][1]))) {
      option_number++;
      option_mask += 1;
    }
  }
```

```c

  if(option_number == 0) {
    return -1;
  }

  choice = random_dir(option_number);
  choice++;

  while(option_counter != choice) {
    if(option_mask % 2 == 1) {
      option_counter++;
    }
    chosen_direction--;
    option_mask /= 2;
  }

  *x = *x + relative_dir[chosen_direction][0];
  *y = *y + relative_dir[chosen_direction][1];

  return chosen_direction;
}

int **initialize_visited(int width, int height) {
  int **visited = (int **)calloc(sizeof(int *), height);
  for(int row = 0; row < height; row++) {
    *(visited + row) = (int *)calloc(sizeof(int), width);
  }
  return visited;
}

int **initialize_parent(int width, int height, int xstart, int ystart) {
  int **parent = (int **)calloc(sizeof(int *), height);
  for(int row = 0; row < height; row++) {
    *(parent + row) = (int *)calloc(sizeof(int), width);
    for(int column = 0; column < width; column++) {
      *(*(parent + row) + column) = -1;
    }
  }
  *(*(parent + xstart) + ystart) = -2;
  return parent;
}

void smash_walls(int x, int y, int direction, int **maze) {
  switch(direction) {
    case 0: //we came from left
      *(*(maze + x) + y - 1) = 0;
      break;
    case 1: //we came from above
      *(*(maze + x - 1) + y) = 0;
      break;
    case 2: //we came from right
      *(*(maze + x) + y + 1) = 0;
      break;
    case 3: //we came from below
      *(*(maze + x + 1) + y) = 0;
      break;
  }
}
```

```c

void iterative_dfs(int xstart, int ystart, int xmax, int ymax, int **maze, int **visited) {
  cell *top = NULL;
  int curr_x = xstart, curr_y = ystart, old_x = xstart, old_y = ystart, direction;

  do {
    direction = direction_choice(&curr_x, &curr_y, xmax, ymax, visited);
    if(direction != -1) {
      *(*(visited + curr_x) + curr_y) = 1;
      top = stack_add(top, old_x, old_y);
      smash_walls(curr_x, curr_y, direction, maze);
    }
    else {
      if(!is_empty(top)) {
        curr_x = top->x;
        curr_y = top->y;
        top = rem(top);
      }
    }
    old_x = curr_x;
    old_y = curr_y;
  } while(!is_empty(top));
}

void recursive_dfs(int x, int y, int xmax, int ymax, int **maze, int **visited) {
  int direction;
  while((direction = direction_choice(&x, &y, xmax, ymax, visited)) != -1) {
    *(*(visited + x) + y) = 1;
    smash_walls(x, y, direction, maze);
    recursive_dfs(x, y, xmax, ymax, maze, visited);
  }
}

int is_valid_direction(int x, int y, int width, int height, int **maze, int **parent) {
  return (in_matrix(x, y, height, width) &&
          (*(*(maze + x) + y) == 0) &&
          (*(*(parent + x) + y) == -1));
}

void bfs(int xstart, int ystart, int width, int height, int **maze, int **parent) {
  cell *front = queue_add(NULL, xstart, ystart);
  cell *back = front;

  while(!is_empty(front)) {
    int curr_x = front->x;
    int curr_y = front->y;

    //we test all the four directions
    for(int direction = 0; direction < 4; direction++) {
      int new_x = curr_x + relative_dir[direction][0] / 2;
      int new_y = curr_y + relative_dir[direction][1] / 2;
      if(is_valid_direction(new_x, new_y, width, height, maze, parent)) {
        *(*(parent + new_x) + new_y) = direction;
        back = queue_add(back, new_x, new_y);
      }
    }
    front = rem(front);
  }
```

```c
}

void clear_mem(int height, int ***matrix) {
  for(int row = 0; row < height; row++) {
    free(*(*matrix + row));
  }
  free(*matrix);
}

int direction_relativization(int current_dir, int next_dir) {
  if(current_dir == next_dir) { //forward
    return 0;
  }
  if((current_dir + 4 - next_dir) % 4 == 1) { //turn left
    return 2;
  }
  //turn right
  return 1;
}

void recover_path(int xstop, int ystop, int **parent, char *file_name) {
  cell *top = NULL;
  int curr_x = xstop, curr_y = ystop;
  int next_direction = 0, actual_direction, previous_direction = 0, previous_rel_dir = 0;
  int length = 2;
  while(*(*(parent + curr_x) + curr_y) != -2) {
    top = stack_add(top, *(*(parent + curr_x) + curr_y), 0);
    actual_direction = (*(*(parent + curr_x) + curr_y) + 2) % 4;
    curr_x = curr_x + relative_dir[actual_direction][0] / 2;
    curr_y = curr_y + relative_dir[actual_direction][1] / 2;
    if(previous_direction == actual_direction)
      length++;
    previous_direction = actual_direction;
  }

  FILE *path_file = fopen(file_name, "w");
  fprintf(path_file, "%d\n", length);
  while(!is_empty(top)) {
    actual_direction = top->x;
    if(actual_direction == -2) {
      actual_direction = 0;
    }
    next_direction = (top->link != NULL) ? top->link->x : 0;

    if(previous_rel_dir == 0) {
      fprintf(path_file, "%d %d %d\n", direction_relativization(actual_direction, next_direction), curr_x, curr_y);
    }
    curr_x = curr_x + relative_dir[actual_direction][0] / 2;
    curr_y = curr_y + relative_dir[actual_direction][1] / 2;
    top = rem(top);
    previous_rel_dir = direction_relativization(actual_direction, next_direction);
  }
  fprintf(path_file, "%d %d %d\n", -1, curr_x, curr_y);
  fclose(path_file);
}

void print_maze(int width, int height, int **maze) {
```

```c
  for(int row = 0; row < height; row++) {
    for(int column = 0; column < width; column++) {
      *(*(maze + row) + column) ? printf("%c", 219) : printf(" ");
    }
    printf("\n");
  }
}

int **initialize_maze(int width, int height) {
  int **maze = (int **)calloc(sizeof(int *), height);
  for(int row = 0; row < height; row++) {
    *(maze + row) = (int *)calloc(sizeof(int), width);
    for(int column = 0; column < width; column++) {
      *(*(maze + row) + column) = ((row % 2 == 1) && (column % 2 == 1)) ? 0 : 1;
    }
  }
  *(*(maze + 1)) = 0;
  *(*(maze + height - 2) + width - 1) = 0;
  return maze;
}

int **generate_maze(int *width, int *height, int seed) {
  (*width) += ((*width) % 2 == 0) ? 1 : 0;
  (*height) += ((*height) % 2 == 0) ? 1 : 0;

  srand(seed);

  int **visited = initialize_visited(*width, *height);
  int **maze = initialize_maze(*width, *height);

  *(*(visited + *height - 2) + *width - 2) = 1;
  iterative_dfs(*height - 2, *width - 2, *height, *width, maze, visited);
  /*
  recursive_dfs(*height - 2, *width - 2, *height, *width, maze, visited);
  */
  clear_mem(*height, &visited);
  return maze;
}

void solve_maze(int width, int height, int **maze, int xstart, int ystart, int xstop, int ystop, char
    *file_name) {
  int **parent = initialize_parent(width, height, xstart, ystart);
  bfs(xstart, ystart, width, height, maze, parent);
  recover_path(xstop, ystop, parent, file_name);
  clear_mem(height, &parent);
}
```

structures.h

```c
#ifndef STRUCTURES_H_INCLUDED
#define STRUCTURES_H_INCLUDED

typedef struct cell {
  int x, y;
  struct cell *link;
} cell;

extern cell *stack_add(cell *old_cell, int cell_x, int cell_y);
extern cell *queue_add(cell *old_cell, int cell_x, int cell_y);
```

```
extern cell *rem(cell *old_cell);
extern int is_empty(cell *top_cell);

#endif
```

structures.c

```
#include <stdlib.h>
#include "structures.h"

cell *stack_add(cell *old_cell, int cell_x, int cell_y) {
  cell *new_cell;
  new_cell = (cell *)malloc(sizeof(cell));
  new_cell->x = cell_x;
  new_cell->y = cell_y;
  new_cell->link = old_cell;
  return new_cell;
}

cell *queue_add(cell *old_cell, int cell_x, int cell_y) {
  cell *new_cell;
  new_cell = (cell *)malloc(sizeof(cell));
  new_cell->x = cell_x;
  new_cell->y = cell_y;
  new_cell->link = NULL;
  if(old_cell != NULL) {
    old_cell->link = new_cell;
  }
  return new_cell;
}

cell *rem(cell *old_cell) {
  cell *new_cell;
  new_cell = old_cell->link;
  free(old_cell);
  return new_cell;
}

int is_empty(cell *top_cell) {
  return (top_cell == NULL);
}
```

The robot controller:

robot_guide.c

```
#include <webots/motor.h>
#include <webots/robot.h>
#include <webots/gps.h>
#include <webots/compass.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "maze.h"
#include "structures.h"

#define TIME_STEP 16
#define SPEED 5
#define RATIO 1.531
#define CELL_WIDTH 0.2
```

```c

const double velocities[3][2] = {{1, 1}, {1, RATIO}, {RATIO, 1}};

int read_path(int **path, double ***absolute_coord) {
  int steps = 0;
  FILE *path_file = fopen("../supervisor/path.txt", "r");
  fscanf(path_file, "%d", &steps);

  (*path) = (int *)calloc(sizeof(int), steps);
  (*absolute_coord) = (double **)calloc(sizeof(double *), steps);
  for(int step = 0; step < steps; step++) {
    *((*absolute_coord) + step) = (double *)calloc(sizeof(double), 2);
    fscanf(path_file, "%d%lf%lf", (*path) + step, *((*absolute_coord) + step) + 1, *((*absolute_coord) + step));
    *(*((*absolute_coord) + step)) = CELL_WIDTH * *(*((*absolute_coord) + step));
    *(*((*absolute_coord) + step) + 1) = (- 1.0) * CELL_WIDTH * *(*((*absolute_coord) + step) + 1);
  }
  fclose(path_file);
  return steps;
}

int exit_condition(int req_step, int req_orientation, int req_sign, int step, double **absolute_coord, double *orientation, double *position) {
  if(req_step == 0) {
    return (req_sign * (position[req_orientation] - absolute_coord[step + 1][req_orientation]) >= 0.001);
  }
  return (fabs(orientation[req_orientation]) <= 0.9999);
}

int main(int argc, char **argv) {
  wb_robot_init();

  WbDeviceTag fr_motor = wb_robot_get_device("front_right_motor");
  WbDeviceTag fl_motor = wb_robot_get_device("front_left_motor");
  WbDeviceTag gps = wb_robot_get_device("gps");
  WbDeviceTag compass = wb_robot_get_device("compass");
  wb_gps_enable(gps, TIME_STEP);
  wb_compass_enable(compass, TIME_STEP);

  double left_speed = SPEED, right_speed = SPEED;
  double *position, *orientation;
  int step = 0, correction = 1;

  wb_motor_set_velocity(fr_motor, 0);
  wb_motor_set_velocity(fl_motor, 0);
  wb_motor_set_position(fl_motor, INFINITY);
  wb_motor_set_position(fr_motor, INFINITY);

  wb_robot_step(TIME_STEP);
  int *path;
  double **absolute_coord;
  int total_steps = read_path(&path, &absolute_coord);
  int req_step, req_orientation, req_sign;

  position = wb_gps_get_values(gps);
  orientation = wb_compass_get_values(compass);

```

```c
  while(path[step] != -1) {
    req_step = path[step];
    req_orientation = (fabs(orientation[0]) > 0.5) ? 1 : 0;
    req_sign = (position[req_orientation] - absolute_coord[step + 1][req_orientation] < 0) ? -1 : 1;
    wb_motor_set_velocity(fr_motor, SPEED * velocities[path[step]][0]);
    wb_motor_set_velocity(fl_motor, SPEED * velocities[path[step]][1]);
    do {
      position = wb_gps_get_values(gps);
      orientation = wb_compass_get_values(compass);
      wb_robot_step(TIME_STEP);
    } while(exit_condition(req_step, req_orientation, req_sign, step, absolute_coord, orientation,
     position));
    step++;
  }

  wb_motor_set_velocity(fr_motor, 0);
  wb_motor_set_velocity(fl_motor, 0);

  wb_robot_cleanup();
  return 0;
}
```

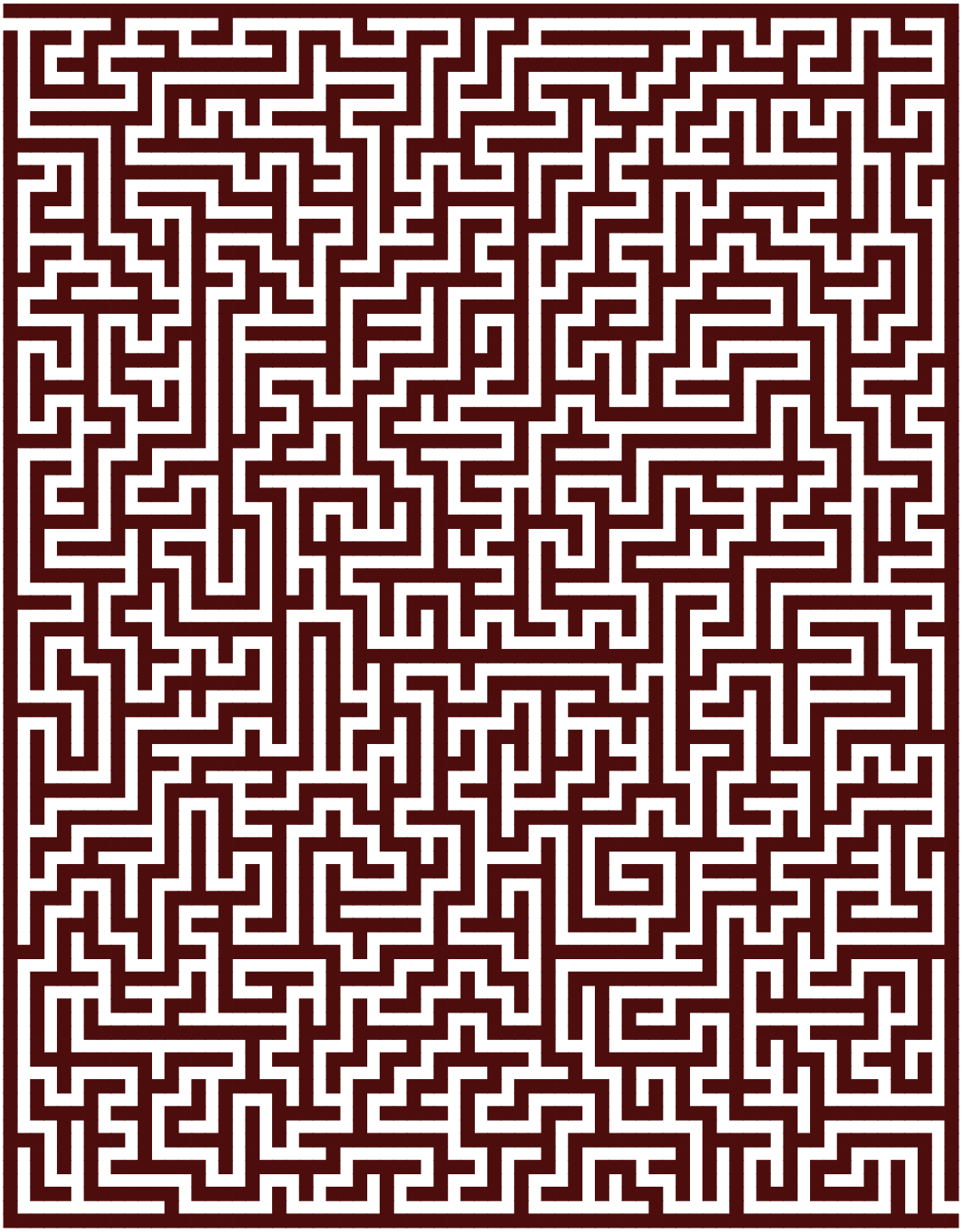
The supervisor of the simulation:

supervisor.c

```c
#include <webots/robot.h>
#include <webots/supervisor.h>
#include <stdio.h>
#include <stdlib.h>
#include "maze.h"
#include "structures.h"

#define TIME_STEP 32
#define CELL_WIDTH 0.2

char position_string[100], name_string[100];
char floor_position_string[100];
void spawn_maze(int **matrix, int rows, int columns, WbFieldRef children_field);


int main(int argc, char **argv) {

  wb_robot_init();

  WbNodeRef root_node = wb_supervisor_node_get_root();
  WbFieldRef children_field = wb_supervisor_node_get_field(root_node, "children");

  int width = 3, height = 3, seed = 23;

  FILE *fp;
  fp = fopen("input.txt", "r");
  if (fp == NULL) {
      printf("Error opening the file.\n");
      return 1;
  }
  fscanf(fp, "%d %d %d", &width, &height, &seed);
  fclose(fp);

  int **maze = generate_maze(&width, &height, seed);
```

```c
  solve_maze(width, height, maze, 1, 0, height - 2, width - 1, "path.txt");

  sprintf(floor_position_string, "Basic_floor {translation %f %f 0, size %f %f}", 0.5 * CELL_WIDTH * (-1 + width), 0.5 * CELL_WIDTH * (1 - height), CELL_WIDTH * width, CELL_WIDTH * height);
  wb_supervisor_field_import_mf_node_from_string(children_field, 2, floor_position_string);

  spawn_maze(maze, height, width, children_field);

  while (wb_robot_step(TIME_STEP) != -1);

  wb_robot_cleanup();
  return 0;
}

void spawn_maze(int **matrix, int rows, int columns, WbFieldRef children_field)
{

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < columns; j++)
        {
            if (matrix[i][j] == 1)
            {
                sprintf(position_string, "wall_proto {translation %f %f %f}", CELL_WIDTH * j, CELL_WIDTH *(-1) * i, CELL_WIDTH * 0.5);
                wb_supervisor_field_import_mf_node_from_string(children_field, -1, position_string);
            }
        }
}
```

B - *A challenge for the reader*



`width` = 71, `height` = 91, `seed` = 1972